SOFTWARE ENGINEERING — UNIT 1

Important 6-Mark Exam Answers

This document contains the complete Unit 1 explanations in exam-ready format. The topics, explanations, examples, diagrams, comparisons, and exam tips are retained in full.

PART A: IMPORTANCE OF SOFTWARE ENGINEERING

1. Importance of Software Engineering

Definition

Software Engineering is a systematic approach to designing, developing, testing, deploying, and maintaining software. It applies engineering principles to software development to produce reliable, efficient, and high-quality software.

Importance of Software Engineering

1. Handles Complexity. Software projects can contain thousands of lines of code and many interconnected components. Software engineering helps organize and manage this complexity.

2. Improves Software Quality. It follows standards, testing procedures, and design principles to develop reliable and error-free software.

3. Reduces Development Cost. Proper planning and early error detection help reduce the cost of fixing mistakes later.

4. Saves Time. A systematic development process helps teams complete projects within the planned schedule.

5. Supports Teamwork. Software engineering defines responsibilities, documentation, and communication methods so that multiple developers can work together.

6. Improves Maintainability. Well-designed software is easier to modify, update, and repair.

7. Increases Customer Satisfaction. Requirement analysis helps developers understand customer needs and deliver software that meets expectations.

Conclusion: Software engineering is important because it helps develop high-quality, reliable, maintainable, and cost-effective software within the required time.

2. Problem-Solving Activity

Definition

Problem-solving is the activity of understanding a real-world problem and developing a software solution to solve it. Software engineers identify the problem, analyze requirements, design a solution, implement it, and test the result.

Steps Involved

1. Problem Identification. Understand the problem that needs to be solved. Example: A college wants to automate student attendance.

2. Requirement Analysis. Identify what the software must do. Example: The system must record attendance, calculate percentages, and generate reports.

3. Solution Design. Plan the structure, algorithms, database, and interfaces required to solve the problem.

4. Implementation. Convert the design into a working program using a programming language.

5. Testing. Check whether the software produces the expected results and identify errors.

6. Evaluation. Verify whether the developed software solves the original problem correctly.

Example

For a college attendance system: identify the attendance problem; collect student and faculty requirements; design the database and user interface; develop the application; test attendance calculations; and deliver the working system.

Conclusion: Problem-solving is a fundamental software engineering activity because it transforms a real-world problem into a practical software solution.

3. Modelling Activity

Definition

Modelling is the process of representing a software system using diagrams, structures, or simplified descriptions before actual implementation. A model helps developers understand how the system should work.

Types of Modelling

1. Requirement Modelling. Represents what the user expects from the system. Example: Use-case diagrams showing the activities of students and faculty.

2. Structural Modelling. Represents the components of a system and their relationships. Example: Class diagrams showing classes such as Student, Teacher, and Course.

3. Behavioural Modelling. Represents how the system behaves when different events occur. Example: A student logs in, views attendance, and logs out.

4. Data Modelling. Represents how information is organized and related. Example: A database model containing Student, Course, and Attendance tables.

Importance of Modelling

- Makes complex systems easier to understand.
- Helps identify design problems early.
- Improves communication between developers and customers.
- Provides a blueprint for implementation.
- Reduces development errors and rework.

Example

Before developing an online shopping application, engineers create models for customers, products, shopping carts, payments, and orders. These models help developers understand the system before writing code.

Conclusion: Modelling provides a clear representation of the software system and helps developers design a correct solution before implementation.

4. Knowledge Acquisition Activity

Definition

Knowledge acquisition is the process of collecting, understanding, organizing, and recording the information needed to develop software. This knowledge can come from customers, domain experts, existing systems, documents, and business rules.

Steps Involved

1. Identify Knowledge Sources. Determine who or what can provide the required information. Example: Teachers, students, and college administrators.

2. Collect Information. Gather information through interviews, questionnaires, observation, and document analysis.

3. Analyze the Information. Understand the collected information and identify important requirements and rules.

4. Organize the Knowledge. Arrange the information into requirements, business rules, data structures, and system behaviour.

5. Document the Knowledge. Record the information in requirement specifications and other project documents.

6. Validate the Knowledge. Confirm with users and experts that the collected information is correct and complete.

Example

While developing a hospital management system, software engineers collect information about patient registration, doctor appointments, medical records, billing, and pharmacy operations. This knowledge is used to develop the correct system.

Importance

- Helps developers understand the application domain.
- Reduces misunderstandings about requirements.
- Prevents missing important features.
- Improves software correctness.
- Supports better design decisions.

Conclusion: Knowledge acquisition helps software engineers understand user needs and domain rules, enabling them to develop software that solves the correct problem.

5. Rationale-Driven Activity

Definition

A rationale-driven activity is a software engineering activity in which developers record the reasons behind important decisions made during software development. It explains why a particular decision was made, not just what decision was made.

Main Elements

1. Decision Identification. Identify a problem or situation that requires a decision.

2. Identify Alternatives. Consider different possible solutions.

3. Evaluate Alternatives. Compare solutions based on cost, performance, security, complexity, and other factors.

4. Select a Solution. Choose the most appropriate alternative.

5. Record the Reason. Document why the selected solution is better than the alternatives.

6. Review the Decision. Reconsider the decision when requirements or circumstances change.

Example

Suppose a developer must choose a database for an online shopping application. Possible alternatives are MySQL, MongoDB, and PostgreSQL. The developer compares them based on data structure, scalability, performance, and project requirements. The developer selects a database and documents the reasons for the choice. This documented reasoning is called the design rationale.

Importance

- Makes design decisions easier to understand.
- Helps future developers maintain the software.
- Avoids repeating previously rejected solutions.
- Supports teamwork and technical discussions.
- Helps evaluate decisions when requirements change.

Conclusion: Rationale-driven activity improves the transparency and maintainability of software projects by recording the reasoning behind important technical decisions.

6. Umbrella Activities in Software Engineering

Definition

Umbrella activities are activities performed throughout the software development process to support and improve the quality of the software project. Unlike activities performed only during a particular phase, umbrella activities continue across multiple phases of development.

Important Umbrella Activities

1. Project Tracking and Control. Monitors project progress and compares actual progress with the planned schedule.

2. Risk Management. Identifies possible problems and prepares plans to reduce their impact. Example: Planning an alternative if a critical technology fails.

3. Software Quality Assurance (SQA). Ensures that software processes and products follow defined quality standards.

4. Technical Reviews. Reviews requirements, designs, and code to identify defects before they become expensive to fix.

5. Software Configuration Management (SCM). Manages changes to source code, documents, and other project components. Example: Using Git to track code changes.

6. Measurement. Collects metrics such as defect counts, development effort, and project progress.

7. Software Documentation. Creates and maintains documents describing requirements, design, testing, and usage.

8. Reusability Management. Identifies and manages software components that can be reused in other projects.

Importance

- Improves software quality.
- Helps control project cost and schedule.
- Reduces project risks.
- Supports change management.
- Improves coordination among team members.

Conclusion: Umbrella activities support all major software engineering activities and help ensure that software is developed systematically and with high quality.

Exam tip: Umbrella activities are especially easy to score marks on if you explain at least six activities with one or two lines each.

7. Software Engineering Challenges

Definition

Software engineering challenges are difficulties faced by developers while designing, developing, testing, deploying, and maintaining software systems.

Major Challenges

1. Complexity. Large software systems contain many modules and interactions, making them difficult to understand and manage.

2. Changing Requirements. Customer needs may change during development. These changes can affect the design, cost, and schedule.

3. Time and Cost Constraints. Software must often be delivered within a fixed budget and deadline.

4. Software Quality. Developing software that is reliable, secure, efficient, and easy to maintain is challenging.

5. Security. Software must protect sensitive information against unauthorized access, attacks, and data leaks.

6. Maintenance. Existing software needs bug fixes, updates, and new features after deployment.

7. Scalability. Software must handle increasing users, transactions, and data without significant performance degradation.

8. Team Communication. Miscommunication among developers, testers, managers, and customers can lead to incorrect requirements and implementation errors.

Example

An online banking application must handle thousands of transactions, protect customer information, maintain accuracy, and remain available. Meeting all these requirements simultaneously creates several engineering challenges.

Conclusion: Software engineering challenges can be addressed through proper planning, requirement analysis, testing, security practices, risk management, and continuous maintenance.

8. Software Development Life Cycle (SDLC)

Definition

The Software Development Life Cycle (SDLC) is a structured process used to develop software through a sequence of phases, from identifying requirements to maintaining the finished product.

Phases of SDLC

1. Planning and Feasibility Study. Determine the project's objectives, estimated cost, required resources, schedule, and feasibility. Feasibility may include technical, economic, operational, and schedule considerations.

2. Requirement Analysis. Collect and analyze customer requirements. The requirements are documented in the Software Requirements Specification (SRS).

3. System Design. Prepare the software architecture, database design, user interfaces, and component specifications.

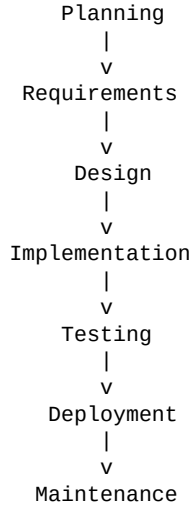
4. Implementation (Coding). Developers convert the design into source code using programming languages.

5. Testing. Test the software to identify defects and verify that it meets the requirements. Common testing levels include unit testing, integration testing, system testing, and acceptance testing.

6. Deployment. Install and release the software for actual users.

7. Maintenance. Fix bugs, improve performance, adapt to environmental changes, and add necessary features after deployment.

SDLC Diagram



Advantages of SDLC

- Provides a systematic development process.
- Improves project planning and management.
- Helps identify errors early.
- Improves software quality.
- Makes project progress easier to monitor.
- Supports proper documentation.

Conclusion: SDLC provides a structured framework for developing, delivering, and maintaining reliable software within the required time and budget.

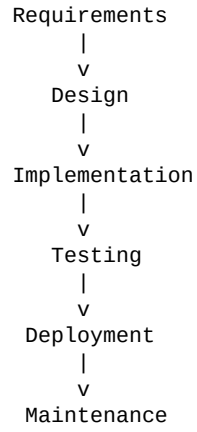
PART B: SOFTWARE PROCESS MODELS

9. Waterfall Model

Definition

The Waterfall Model is a traditional software development model in which development proceeds through a sequence of phases. Each phase is generally completed before the next phase begins. It is called the waterfall model because progress flows downward from one phase to another.

Diagram



Phases

1. Requirements Analysis. All requirements are collected and documented before development begins.

2. System Design. The architecture, database, and system components are designed.

3. Implementation. Developers write the program according to the design.

4. Testing. The completed software is tested to find defects and verify requirements.

5. Deployment. The software is delivered to users.

6. Maintenance. Errors are corrected and necessary updates are provided.

Advantages

- Simple and easy to understand.
- Clear phases and milestones.
- Easy to manage when requirements are stable.
- Provides clear documentation.
- Suitable for small projects with well-defined requirements.

Disadvantages

- Difficult to accommodate changing requirements.
- Working software becomes available relatively late.
- Testing occurs after implementation.
- Errors discovered late may be expensive to correct.
- Limited customer feedback during development.

Example

A small government record-management system with fixed, clearly documented requirements may be developed using the Waterfall Model.

Conclusion: The Waterfall Model is suitable for projects with stable and well-understood requirements, but it is less suitable when frequent changes are expected.

10. Incremental Model

Definition

The Incremental Model develops software in multiple smaller parts called increments. Each increment adds new functionality to the existing working software. Instead of delivering the entire system at once, developers release it in stages.

Diagram

```
Requirements
     |
     v
Overall Design
     |
     v
Increment 1 -> Core Features
     |
     v
Increment 2 -> Additional Features
     |
     v
Increment 3 -> More Features
     |
     v
 Complete Software
```

Working

1. Requirement Analysis. Identify the overall system requirements and prioritize the features.

2. Overall Design. Prepare a high-level design for the complete system.

3. First Increment. Develop and test the most important features.

4. Subsequent Increments. Add more features in separate development cycles.

5. Integration and Testing. Ensure that each new increment works correctly with existing functionality.

6. Final Delivery. Deliver the complete system after all planned increments are implemented.

Example

An online shopping application can be developed incrementally: Increment 1—User registration and login; Increment 2—Product catalogue; Increment 3—Shopping cart and ordering; Increment 4—Online payment and order tracking. Users can start using the core features before all features are completed.

Advantages

- Working software is delivered early.
- Important features can be developed first.

- Customer feedback can guide later increments.
- Testing and debugging are easier for smaller parts.
- Changes can be incorporated into later increments.

Disadvantages

- Requires good overall architecture.
- Integration between increments can be difficult.
- Planning and managing increments requires effort.
- Not every system can be divided easily into independent increments.

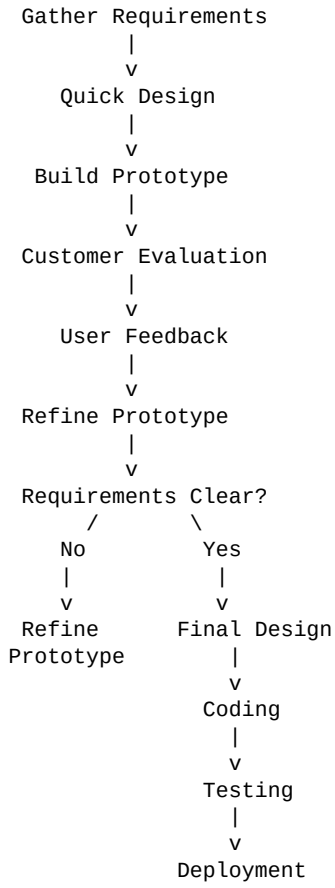
Conclusion: The Incremental Model is useful when software can be divided into smaller functional parts and early delivery is important.

11. Prototype Model

Definition

The Prototype Model develops an early, simplified version of software to understand and validate customer requirements before developing the final system. A prototype helps customers visualize the system and provide feedback.

Diagram



Phases

1. Requirement Collection. Collect the initial requirements from the customer.

2. Quick Design. Create a simple design focusing on the main features and user interface.

3. Prototype Development. Build a preliminary version of the system.

4. Customer Evaluation. Allow users to examine the prototype and provide feedback.

5. Refinement. Modify the prototype according to the feedback.

6. Final Development. Once the requirements are sufficiently clear, develop, test, and deploy the actual software.

Example

A company wants to develop a mobile banking application but is unsure about the best interface. Developers create a prototype containing login, account balance, and transaction screens. Customers review it and suggest changes before the actual application is developed.

Advantages

- Helps clarify unclear requirements.
- Encourages customer involvement.
- Identifies usability problems early.
- Reduces the risk of developing the wrong product.
- Improves understanding between customers and developers.

Disadvantages

- Repeated changes may increase development time.
- Customers may mistake the prototype for the finished product.
- Developers may use poor design practices to build the prototype quickly.
- Excessive refinement can increase cost.

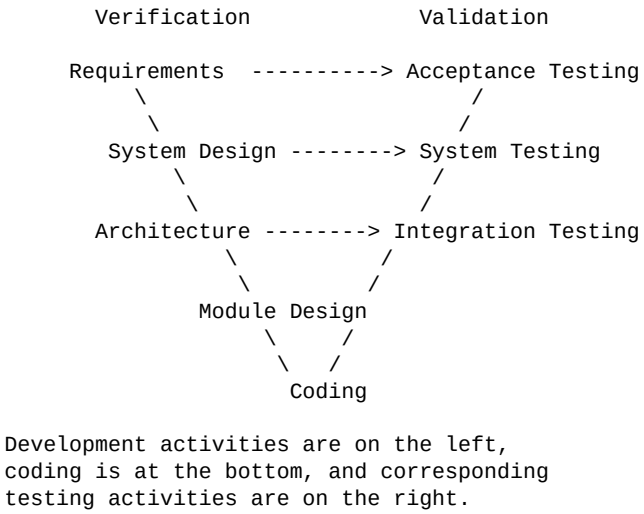
Conclusion: The Prototype Model is particularly useful when customer requirements are unclear or the user interface needs frequent evaluation.

12. V-Model

Definition

The V-Model, also called the Verification and Validation Model, is a software development model in which development phases are associated with corresponding testing phases. It is called the V-Model because its phases are commonly represented in the shape of the letter V.

Diagram



Development activities are on the left,
coding is at the bottom, and corresponding
testing activities are on the right.

Phases

1. Requirement Analysis. Understand customer needs. These requirements are later validated through acceptance testing.

2. System Design. Define the overall system structure. System testing checks the complete system against the specified requirements.

3. Architectural Design. Identify the major modules and their interactions. Integration testing checks communication between these modules.

4. Module Design. Design the internal logic of individual modules. Unit testing checks the individual modules.

5. Coding. Implement the software based on the completed designs.

6. Testing. Perform the corresponding testing activities to verify that the developed system satisfies its specifications.

Verification and Validation

- **Verification:** Are we building the product correctly according to its specifications?
- **Validation:** Are we building the correct product that meets the user's needs?

Advantages

- Testing is planned early.
- Development phases are clearly defined.
- Defects can be identified systematically.
- Suitable for projects with stable requirements.
- Easy to manage and monitor.

Disadvantages

- Difficult to accommodate changing requirements.
- Working software is generally available late.
- No early working prototype is required by the model.
- Changes to earlier phases may be costly.

Example

The V-Model may be appropriate for medical-device software where requirements are stable and systematic verification and validation are essential.

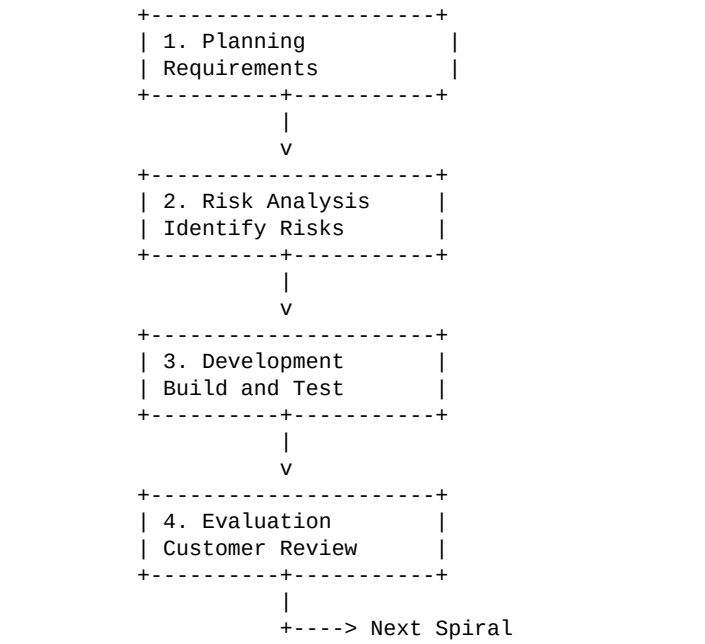
Conclusion: The V-Model emphasizes systematic testing and is suitable for projects where requirements are clear and quality assurance is critical.

13. Spiral Model

Definition

The Spiral Model is a risk-driven software development model that combines iterative development with systematic risk analysis. The software is developed through repeated cycles called spirals. Each cycle includes planning, risk analysis, development, and evaluation.

Diagram




```
In an actual Spiral Model diagram, the cycles
expand outward to represent increasing project
effort or progress.
```

Phases of Each Spiral

1. Planning. Determine the objectives, requirements, alternatives, and constraints for the current cycle.

2. Risk Analysis. Identify and evaluate potential risks. Develop strategies to reduce major risks.

3. Engineering. Design, develop, and test the software or the next increment.

4. Customer Evaluation. Obtain feedback, evaluate the results, and plan the next spiral.

Example

A large banking system may use the Spiral Model because security risks, integration challenges, and performance requirements need to be evaluated throughout development.

Advantages

- Strong emphasis on risk management.
- Supports changing requirements.
- Allows repeated customer feedback.
- Suitable for large and complex projects.
- Problems can be identified during successive cycles.

Disadvantages

- Expensive to implement.
- Requires expertise in risk analysis.
- Management can be complex.
- Unsuitable for small projects with low risk.

Conclusion: The Spiral Model is best suited for large, complex, high-risk projects where risks must be evaluated continuously throughout development.

14. Rapid Application Development (RAD) Model

Definition

Rapid Application Development (RAD) is a software process model that emphasizes rapid development, reusable components, prototyping, user involvement, and frequent delivery. Its main goal is to develop working software quickly without sacrificing necessary quality.

Phases

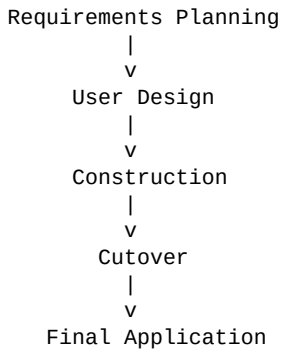
1. Requirements Planning. Developers and customers identify the project's objectives, requirements, and scope.

2. User Design. Users and developers collaborate to create prototypes and refine the system design.

3. Construction. Developers build the software using reusable components, development tools, and iterative improvements.

4. Cutover. The system is tested, converted to the production environment, deployed, and handed over to users.

Diagram



Example

A company needs an internal employee-management application quickly. Using RAD, developers create prototypes, obtain feedback from employees, reuse existing components, and rapidly build modules for employee records, leave management, and reporting.

Advantages

- Enables faster development.
- Encourages continuous user involvement.
- Supports frequent feedback and changes.
- Promotes component reuse.
- Working software can be delivered quickly.

Disadvantages

- Requires skilled developers.
- Requires continuous customer participation.
- May not be suitable for very large or highly complex systems.
- Needs tools and resources that support rapid development.
- Poor coordination can create integration problems.

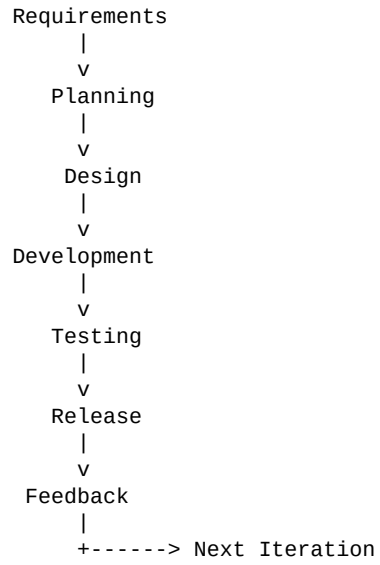
Conclusion: RAD is suitable for projects requiring rapid delivery, active customer participation, and software that can be divided into modules.

15. Agile Model

Definition

Agile is an iterative and incremental software development approach that delivers working software in small releases and adapts to changing customer requirements. It emphasizes collaboration, customer feedback, continuous improvement, and frequent delivery.

Diagram



Working of Agile

1. Requirement Collection. Identify and prioritize features based on customer needs.

2. Iteration Planning. Select a small set of features to develop during the next iteration or sprint.

3. Design and Development. Design and implement the selected features.

4. Testing. Test the developed features to ensure they work correctly.

5. Release. Deliver the working software or increment to the customer.

6. Feedback and Improvement. Collect customer feedback and use it to plan the next iteration.

Example

A food-delivery application can be developed using Agile: Sprint 1—User registration and login; Sprint 2—Restaurant listing; Sprint 3—Food ordering; Sprint 4—Payment integration; Sprint 5—Order tracking. At the end of each sprint, the team reviews the work and adjusts future plans based on feedback.

Advantages

- Easily accommodates changing requirements.
- Delivers working software frequently.
- Encourages customer collaboration.
- Identifies defects through continuous testing.
- Improves communication among team members.
- Provides opportunities for continuous improvement.

Disadvantages

- Frequent changes may affect cost and schedule.
- Requires regular customer involvement.
- Final cost and delivery dates may be difficult to predict when scope changes frequently.

- Documentation may be insufficient if neglected.
- Requires effective teamwork and communication.

Conclusion: Agile is suitable for projects where requirements change frequently and customers need regular releases of working software.

PART C: IMPORTANT COMPARISONS FOR EXAMS

These comparisons are useful if your examiner asks a question such as “Differentiate between the following models.”

16. Waterfall Model vs Incremental Model

Waterfall Model	Incremental Model
Development proceeds through sequential phases.	Development proceeds through multiple increments.
The complete product is generally delivered after all phases are completed.	Working software is delivered in stages.
Changes to requirements are difficult to accommodate.	Changes can often be included in later increments.
Customer feedback during development is generally limited.	Feedback can be collected after each increment.
Suitable for stable requirements.	Suitable when features can be delivered in stages.

Key difference: Waterfall develops the system mainly as one complete sequence, while Incremental delivers it through multiple functional releases.

17. Prototype Model vs Spiral Model

Prototype Model	Spiral Model
Focuses on understanding and refining requirements.	Focuses on identifying and managing risks.
Builds an early version of the proposed system.	Develops the system through repeated risk-driven cycles.
Customer feedback is used to refine the requirements and design.	Customer evaluation helps guide the next spiral.
Useful when requirements are unclear.	Useful when project risks are high.
Repeated prototyping may increase cost.	Risk analysis and repeated cycles may increase cost.

Key difference: Prototyping primarily clarifies what the customer wants, whereas Spiral places particular emphasis on managing project risks.

18. Waterfall Model vs V-Model

Waterfall Model	V-Model
Uses sequential development phases.	Uses development phases associated with corresponding testing phases.
Testing is generally performed after implementation.	Testing is planned alongside development activities.
Testing activities are not explicitly paired with each development phase in the basic model.	Unit, integration, system, and acceptance testing are associated with development
Suitable for stable requirements.	Suitable for stable requirements requiring systematic verification and validation.
Emphasizes sequential development.	Emphasizes sequential development and structured testing.

Key difference: The V-Model explicitly connects development activities with corresponding testing activities.

19. RAD Model vs Agile Model

RAD Model	Agile Model
Emphasizes rapid development using prototyping and reusable components.	Emphasizes iterative development, collaboration, and adaptation.
Focuses strongly on reducing development time.	Focuses on delivering value continuously and responding to change.
Often relies on specialized tools and reusable components.	Can use various tools and engineering practices.
Requires close user involvement during design.	Requires ongoing customer collaboration and feedback.
Works well when the project can be modularized and developed rapidly.	Works well when requirements evolve over time.

Key difference: RAD focuses strongly on rapid construction, while Agile emphasizes iterative delivery, collaboration, and adaptability.

PART D: WHICH TOPICS SHOULD YOU PREPARE FIRST?

20. Priority Order for Semester Examination

Priority	Topic	What to remember
1	SDLC	Definition, seven phases, diagram, advantages
2	Waterfall Model	Phases, diagram, advantages, disadvantages
3	Incremental Model	Meaning, increments, example, pros and cons
4	Spiral Model	Four activities, risk analysis, diagram
5	Agile Model	Iterations, feedback, advantages, disadvantages
6	V-Model	Diagram, verification, validation, testing levels
7	Prototype Model	Prototype cycle, feedback, applications
8	RAD Model	Four phases, rapid development, limitations
9	Umbrella Activities	Explain at least six activities
10	Software Engineering Challenges	Explain at least six challenges
11	Problem-Solving Activity	Steps and example
12	Modelling Activity	Types and importance
13	Knowledge Acquisition	Steps and example
14	Rationale-Driven Activity	Decisions, alternatives, justification

How to Write a 6-Mark Answer

For most of these questions, use this structure:

1. Definition: 1 mark.

2. Explanation or phases: 2–3 marks.

3. Diagram or example: 1 mark.

4. Advantages, disadvantages, or importance: 1–2 marks.

The exact marking scheme depends on your university, but this structure helps you cover the important points.

Final exam tip: Practise drawing the Waterfall, Incremental, Prototype, V-Model, Spiral, and Agile diagrams. For process-model questions, a correct diagram combined with a definition, working steps, advantages, disadvantages, and an example makes your answer more complete.